GATE in Data Science and AI Study Materials
Data Structures using Python
By Piyush Wairale

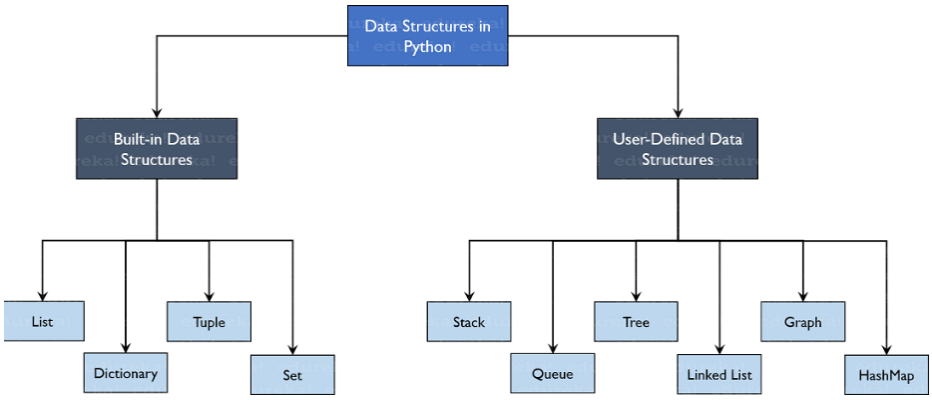
Instructions:

- Kindly go through the lectures/videos on our website www.piyushwairale.com
- Read this study material carefully and make your own handwritten short notes. (Short notes must not be more than 5-6 pages)
- Attempt the question available on portal.
- Revise this material at least 5 times and once you have prepared your short notes, then revise your short notes twice a week
- **If you are not able to understand any topic or required detailed explanation, please mention it in our discussion forum on webiste**
- **Let me know, if there are any typos or mistake in study materials. Mail me at piyushwairale100@gmail.com**

For GATE DA Crash Course, visit: www.piyushwairale.com

1 Data Structures in Python

Data Structures are a way of organizing data so that it can be accessed more efficiently depending upon the situation. Data Structures are fundamentals of any programming language around which a program is built. Python helps to learn the fundamental of these data structures in a simpler way as compared to other programming languages.



1.1 Built-In Data Structures

- Built-in data structures in Python can be divided into two broad categories: mutable and immutable.
- Mutable data structures are those which we can modify – for example, by adding, removing, or changing their elements. Python has three mutable data structures: lists, dictionaries, and sets.
- Immutable data structures, on the other hand, are those that we cannot modify after their creation. The only basic built-in immutable data structure in Python is a tuple.

1.1.1 List

- Lists are dynamic, ordered collections of elements that can hold items of various data types, including numbers, strings, and even other lists. Their defining feature is their ability to be modified, expanded, or reduced during program execution. This versatility makes lists immensely useful for scenarios where data needs to be managed flexibly.
- Lists are created using square brackets, and elements within them are separated by commas.
- Lists are mutable, ordered sequences of elements in Python. Declared using square brackets `[ ]`.
- List items are ordered, changeable, and allow duplicate values.

For GATE DA Crash Course, visit: www.piyushwairale.com

- List items are indexed, the first item has index [0], the second item has index [1] etc.
- When we say that lists are ordered, it means that the items have a defined order, and that order will not change. If you add new items to a list, the new items will be placed at the end of the list.
- The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.

1.1.2 Operations:

1. **Creation:**

```
my_list = [1, 2, 3, 'a', 'b']
```

2. **Accessing Elements:**
   - Indexing: my_list[0] returns the first element.
   - Slicing: my_list[1:3] returns a sublist from index 1 to 2.
3. **Adding Elements:**
   - my_list.append(4) adds 4 to the end.
   - my_list.insert(1, 'x') inserts 'x' at index 1.
4. **Removing Elements:**
   - my_list.remove('a') removes the first occurrence of 'a'.
   - my_list.pop(2) removes the element at index 2.
5. **Other Operations:**
   - len(my_list) returns the length.
   - my_list + [5, 6] concatenates lists.

For GATE DA Crash Course, visit: www.piyushwairale.com

1.2 Tuples

- Tuples resemble lists in that they are ordered collections of elements, but they have a significant difference: they are immutable, meaning their content cannot be changed after creation.
- Tuples are defined using parentheses, and elements are separated by commas.
- Tuples are immutable, ordered sequences. Declared using parentheses `( )`.
- Tuple items are ordered, unchangeable, and allow duplicate values.
- Tuple items are indexed, the first item has index [0], the second item has index [1] etc.
- When we say that tuples are ordered, it means that the items have a defined order, and that order will not change.
- Tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created.

1.2.1 Operations:

1. **Creation:**

```
my_tuple = (1, 2, 'a', 'b')
```

2. **Accessing Elements:** Similar to lists, using indexing and slicing.
3. **Immutable Nature:** Elements cannot be added or removed after creation.

For GATE DA Crash Course, visit: www.piyushwairale.com

1.3 Sets

- Set items are unordered, unchangeable, and do not allow duplicate values.
- Sets are unordered collections of unique elements. Declared using curly braces { } or `set()`.
- Set items are unordered, unchangeable, and do not allow duplicate values.
- Set items are unchangeable, meaning that we cannot change the items after the set has been created.
- Once a set is created, you cannot change its items, but you can remove items and add new items.
- Sets cannot have two items with the same value.

1.3.1 Operations:

1. **Creation:**

```
my_set = {1, 2, 3, 'a', 'b'}
```

2. **Adding and Removing Elements:**
   - my_set.add(4) adds 4 to the set.
   - my_set.remove('a') removes 'a'.
3. **Set Operations:**
   - Union: `set1 | set2`.
   - Intersection: `set1 & set2`.
   - Difference: `set1 - set2`.

For GATE DA Crash Course, visit: www.piyushwairale.com

1.4 Dictionaries

- Dictionaries are used to store data values in key:value pairs.
- A dictionary is a collection which is ordered*, changeable and do not allow duplicates.
- Dictionaries are key-value pairs. Declared using curly braces { } with key-value pairs.
- Dictionary items are ordered, changeable, and does not allow duplicates.
- Dictionary items are presented in key:value pairs, and can be referred to by using the key name
- When we say that dictionaries are ordered, it means that the items have a defined order, and that order will not change.

  Unordered means that the items does not have a defined order, you cannot refer to an item by using an index.

  As of Python version 3.7, dictionaries are ordered. In Python 3.6 and earlier, dictionaries are unordered.
- Dictionaries are changeable, meaning that we can change, add or remove items after the dictionary has been created.
- Dictionaries cannot have two items with the same key

1.4.1 Operations:

1. **Creation:**

```
my_dict = {'key1': 'value1', 'key2': 2, 'key3': [1, 2, 3]}
```

2. **Accessing and Modifying Elements:**
   - my_dict['key1'] returns 'value1'.
   - my_dict['key2'] = 3 modifies the value of 'key2'.
3. **Dictionary Operations:**
   - my_dict.keys() returns keys.
   - my_dict.values() returns values.
   - my_dict.items() returns key-value pairs.

For GATE DA Crash Course, visit: www.piyushwairale.com

1.5 Comparison of List, Tuple, Set, and Dictionary in Python

1. **Homogeneity:**
   - **List:** Non-homogeneous type, can store various elements.
   - **Tuple:** Non-homogeneous type, stores various elements.
   - **Set:** Non-homogeneous type, stores various elements in a single row.
   - **Dictionary:** Non-homogeneous type, stores key-value pairs.
2. **Representation:**
   - **List:** Represented by [ ].
   - **Tuple:** Represented by ( ).
   - **Set:** Represented by { }.
   - **Dictionary:** Represented by { }.
3. **Duplicate Elements:**
   - **List and Tuple:** Allow duplicate elements.
   - **Set:** Does not allow duplicate elements.
   - **Dictionary:** Keys are not duplicated.
4. **Nested Among All:**
   - **List:** Can be nested.
   - **Tuple:** Can be nested.
   - **Set:** Can be nested.
   - **Dictionary:** Can be nested.
5. **Example:**
   - **List:** [6, 7, 8, 9, 10].
   - **Tuple:** (6, 7, 8, 9, 10).
   - **Set:** {6, 7, 8, 9, 10}.
   - **Dictionary:** {6, 7, 8, 9, 10}.
6. **Function for Creation:**
   - **List:** `list()` function.
   - **Tuple:** `tuple()` function.
   - **Set:** `set()` function.

For GATE DA Crash Course, visit: www.piyushwairale.com

- **Dictionary:** `dict()` function.

7. **Mutation:**
   - **List:** Mutable.
   - **Tuple:** Immutable.
   - **Set:** Mutable.
   - **Dictionary:** Mutable, but keys are not duplicated.
8. **Order:**
   - **List and Tuple:** Ordered.
   - **Set:** Unordered.
   - **Dictionary:** Ordered.
9. **Empty Elements:**
   - **List:** `l = []`.
   - **Tuple:** `t = ()`.
   - **Set:** `a = set()` or `b = set(a)`.
   - **Dictionary:** `d = {}`.

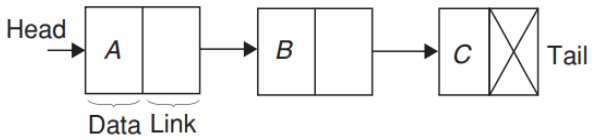
For GATE DA Crash Course, visit: www.piyushwairale.com

2 User-Defined Data Structures

User-defined data structures are not inbuilt in Python, but we can still implement them. We can use the existing functional options in Python to create new data structures. For example, when we say a list = [], Python recognizes it as a list and calls everything related to a list. But when we say a linked list or a queue, Python won't know what these are.

2.1 Linked Lists in Python

- A linked list is a linear data structure consisting of nodes where each node points to the next node in the sequence. It does not have a fixed size and is dynamic in nature.
- Generally node of linked list is represented as a self-referential structure.
- The linked list elements are accessed with special pointer(s) called head and tail.
- A linked list node consists of two parts:
  - **Data:** Holds the value of the node.
  - **Next (and Previous for Doubly Linked List):** Points to the next (and previous) node in the sequence.



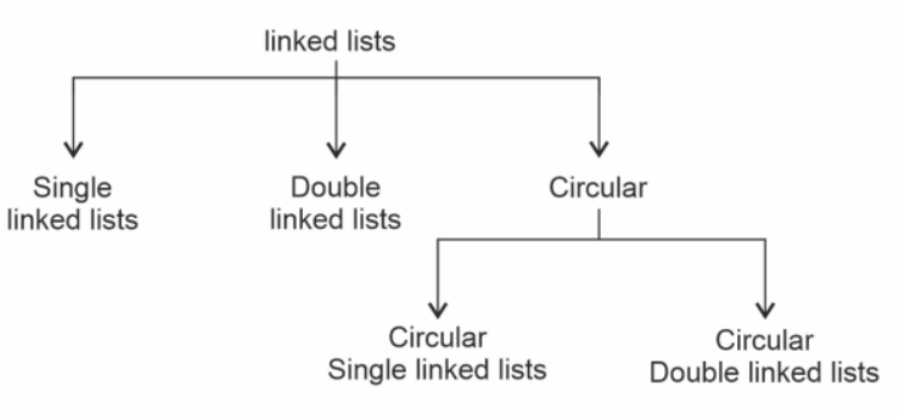
- The principal benefit of a linked list over a conventional array is that the list elements can easily be added or removed without reallocation or reorganization of the entire structure because the data items need not be stored contiguously in memory or on disk.
- Linked lists allow insertion and removal of nodes at any point in the list.
- Finding a node that contains a given data, or locating the place where a new node should be inserted may require scanning most or all of the list elements.
- The list element does not have to occupy contiguous memory.
- Adding, insertion or deletion of list elements can be accomplished with minimal disruption of neighbouring nodes

Important points about Linked lists:

- A linked list is an ordered collection of elements.

For GATE DA Crash Course, visit: www.piyushwairale.com

- A linked list is also used to implement other user-defined data structures like stack and queue.
- Using the collections module in Python, we can use the deque object to implement operations like insert and delete on linked lists.
- The first node in a linked list is the head, and we must start all the operations on the linked list from it.
- The last node of the linked list refers to None showing that the linked list is complete.



2.2 Types of Linked Lists:

2.2.1 Singly Linked List:

Each node points to the next node in the sequence.



2.2.2 Doubly Linked List:

- In a double-linked list, every node will have three sections. Head holds the reference of the first node, the "previous" section of the first node holds None, and the next field of the last node refers to None.
- Each node will hold two references along with the data, one to its previous node and the next to the succeeding node.
- Each node points to both the next and the previous nodes.

For GATE DA Crash Course, visit: www.piyushwairale.com

- We add a pointer to the previous node in a doubly-linked list. Thus, we can go in either direction: forward or backward.



- The operations which can be performed in SLL can also be preformed on DLL.
- The major difference is that we have to adjust double reference as compared to SLL.
- We can traverse or display the list elements in forward as well as in reverse direction.

2.2.3 Circular Single Linked Lists

Circular-linked list is completely same as SLL, except, in CLL the last (Tail) node points to first (Head) node of list.

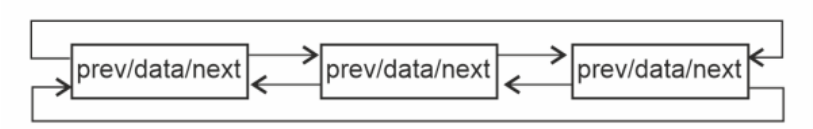
So, the Insertion and Deletion operation at Head and Tail are little different from SLL.



2.2.4 Circular Double Linked Lists

Double circular-linked list can be traversed in both directions again and again. DCL is very similar to DLL, except the last node's next pointer points to first node of list and first node's previous pointer points to last node of list.

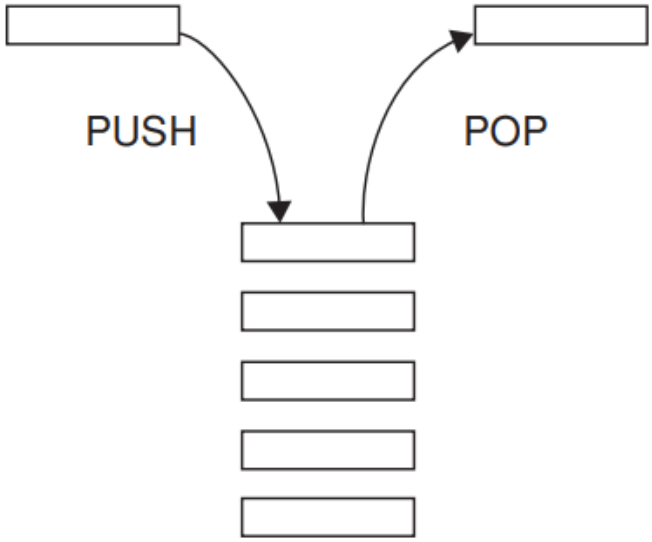
So, the insertion and deletion operations at head and tail in DCL are little different in adjusting the reference as compared to DLL.



For GATE DA Crash Course, visit: www.piyushwairale.com

2.3 Stack

- A stack is a last in first out (LIFO) abstract data type and data structure. A stack can have any abstract data type as an element, but is characterized by only two fundamental operations, **PUSH and POP**
- The PUSH operation adds an item to the top of the stack, hiding any items already on the stack or initializing the stack if it is empty.
- The POP operation removes an item from the top of the stack, and returns the poped value to the caller.
- Elements are removed from the stack in the reverse order to the order of their insertion. Therefore, the lower elements are those that have been on the stack for the longest period.



2.3.1 The functions associated with stack are:

- empty() – Returns whether the stack is empty – Time Complexity: O(1)
- size() – Returns the size of the stack – Time Complexity: O(1)
- top() / peek() – Returns a reference to the topmost element of the stack – Time Complexity: O(1)
- push(a) – Inserts the element 'a' at the top of the stack – Time Complexity: O(1)
- pop() – Deletes the topmost element of the stack – Time Complexity: O(1)

For GATE DA Crash Course, visit: www.piyushwairale.com

2.3.2 Implementation of Stack

Python offers various ways to implement the stack. In this section, we will discuss the implementation of the stack using Python and its module.

We can implement a stack in Python in the following ways.

- **Using List**

  Python list can be used as the stack. It uses the append() method to insert elements to the list where stack uses the push() method. The list also provides the pop() method to remove the last element, but there are shortcomings in the list. The list becomes slow as it grows.

  The list stores the new element in the next to other. If the list grows and out of a block of memory then Python allocates some memory.

- **Using collection.deque**

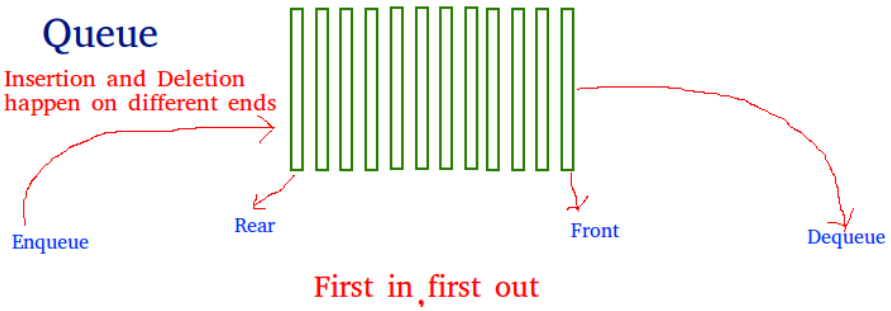
  The collection module provides the deque class, which is used to creating Python stacks. The deque is pronounced as the "deck" which means "double-ended queue". The deque can be preferred over the list because it performs append and pop operation faster than the list. The time complexity is O(1), where the list takes O(n).

- **Using queue module**

  The queue module has the LIFO queue, which is the same as the stack. Generally, the queue uses the put() method to add the data and the () method to take the data.

2.4 Queue

A queue is an ordered collection of items from which items may be deleted at one end (called the front of queue) and into which items may be inserted at the other end (called rear of queue). Queue is a linear data structure that maintains the data in first infirst out (FIFO) order



For GATE DA Crash Course, visit: www.piyushwairale.com

2.4.1 Operations associated with queue are:

- **Enqueue:** Adds an item to the queue. If the queue is full, then it is said to be an Overflow condition – Time Complexity : O(1)
- **Dequeue:** Removes an item from the queue. The items are popped in the same order in which they are pushed. If the queue is empty, then it is said to be an Underflow condition – Time Complexity : O(1)
- **Front:** Get the front item from queue – Time Complexity : O(1)
- **Rear:** Get the last item from queue – Time Complexity : O(1)

2.4.2 Implement a Queue in Python

1. **Using list**

   List is a Python's built-in data structure that can be used as a queue. Instead of enqueue() and dequeue(), append() and pop() function is used. However, lists are quite slow for this purpose because inserting or deleting an element at the beginning requires shifting all of the other elements by one, requiring O(n) time.

2. **Using collections.deque**

   Queue in Python can be implemented using deque class from the collections module. Deque is preferred over list in the cases where we need quicker append and pop operations from both the ends of container, as deque provides an O(1) time complexity for append and pop operations as compared to list which provides O(n) time complexity. Instead of enqueue and deque, append() and popleft() functions are used.

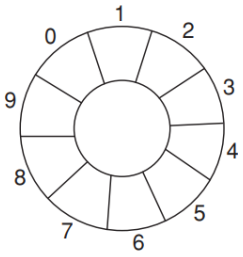
3. **Using queue.Queue**

   Queue is built-in module of Python which is used to implement a queue. queue.Queue(maxsize) initializes a variable to a maximum size of maxsize. A maxsize of zero '0' means a infinite queue. This Queue follows FIFO rule.

For GATE DA Crash Course, visit: www.piyushwairale.com

2.4.3 Circular Queue

As the items from a queue get deleted, the space for that item is reclaimed. Those queue positions continue to be empty. This problem is solved by circular queues. Instead of using a linear approach, a circular queue takes a circular approach; this is why a circular queue does not have a beginning or end.



The advantage of using a circular queue over linear queue is efficient usage of memory.

2.4.4 Priority Queue

- In priority queue, the intrinsic ordering of elements does determine the results of its basic operations. There are two types of priority queues.
- Ascending priority queue is a collection of items in which items can be inserted arbitrarily and from which only the smallest items can be removed.
- Descending priority queue is similar but allows deletion of the largest item.

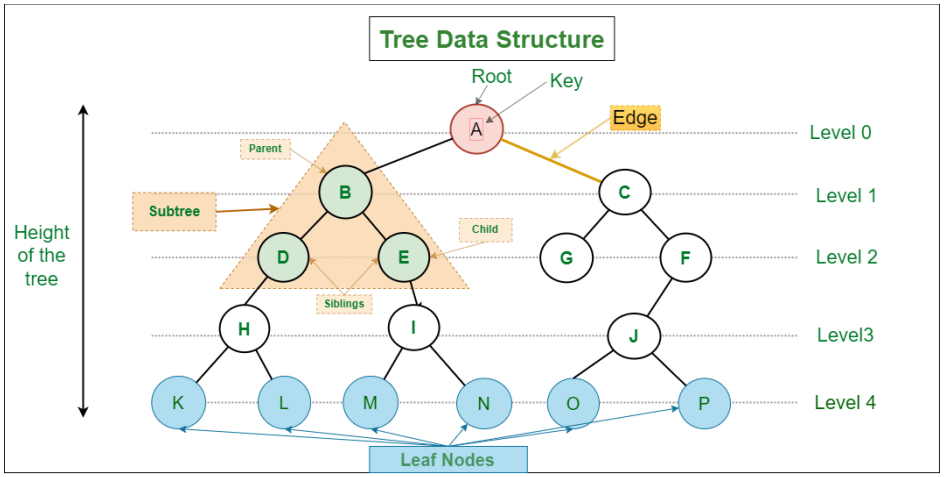
2.5 Trees

- Tree is non-linear data structure designated at a special node called root and elements are arranged in levels without containing cycles.
  (or)
  The tree is
  1. Rooted at one vertex
  2. Contains no cycles
  3. There is a sequence of edges from any vertex to any other
  4. Any number of elements may connect to any node (including root)
  5. A unique path traverses from root to any node of tree
  6. Tree stores data in hierarchical manner
  7. The elements are arranged in layers

- A tree data structure is a hierarchical structure that is used to represent and organize data in a way that is easy to navigate and search. It is a collection of nodes that are connected by edges and has a hierarchical relationship between the nodes.

For GATE DA Crash Course, visit: www.piyushwairale.com

- The topmost node of the tree is called the root, and the nodes below it are called the child nodes. Each node can have multiple child nodes, and these child nodes can also have their own child nodes, forming a recursive structure.
- This data structure is a specialized method to organize and store data in the computer to be used more effectively. It consists of a central node, structural nodes, and sub-nodes, which are connected via edges. We can also say that tree data structure has roots, branches, and leaves connected with one another.



Basic Terminologies In Tree Data Structure:

- **Parent Node:** The node which is a predecessor of a node is called the parent node of that node. B is the parent node of D, E.
- **Child Node:** The node which is the immediate successor of a node is called the child node of that node. Examples: D, E are the child nodes of B.
- **Root Node:** The topmost node of a tree or the node which does not have any parent node is called the root node. A is the root node of the tree. A non-empty tree must contain exactly one root node and exactly one path from the root to all other nodes of the tree.
- **Leaf Node or External Node:** The nodes which do not have any child nodes are called leaf nodes. K, L, M, N, O, P, G are the leaf nodes of the tree.
- **Ancestor of a Node:** Any predecessor nodes on the path of the root to that node are called Ancestors of that node. A,B are the ancestor nodes of the node E
- **Descendant:** Any successor node on the path from the leaf node to that node. E,I are the descendants of the node B.
- **Sibling:** Children of the same parent node are called siblings. D,E are called siblings.

For GATE DA Crash Course, visit: www.piyushwairale.com

- **Level of a node:** The count of edges on the path from the root node to that node. The root node has level 0.
- **Internal node:** A node with at least one child is called Internal Node.
- **Neighbour of a Node:** Parent or child nodes of that node are called neighbors of that node. Subtree: Any node of the tree along with its descendant.

2.5.1 Binary Tree

It is a special type of tree where each node of tree contains either 0 or 1 or 2 children. (or) Binary Tree is either empty, or it consists of a root with two binary trees called left-sub tree and right sub-tree of root (left or right or both the sub trees may be empty)

Properties of Binary Tree

- Binary tree partitioned into three parts.
- First subset contains root of tree.
- Second subset is called left subtree.
- Another subset is called right subtree.
- Each subtree is a binary tree.
- Degree of any node is 0/1/2.
- The maximum number of nodes in a tree with height 'h' is $2^{h+1}-1$.
- The maximum number of nodes at level 'i' is 2^{i-1}.
- For any non-empty binary tree, the number of terminal nodes with n_2, nodes of degree 2 is $N_0 = n_2 + 1$
- The maximum number of nodes in a tree with depth d is $2^d - 1$.

Full binary tree: It is a binary tree, for which all leaf nodes are at same level and all intermediate nodes contains exactly 2 children.
(or) A tree with depth 'K' contains exactly 2K – 1 nodes.

Strictly binary tree: A binary tree in which every node contains exactly 0 or 2 children.

Skewed binary tree: A binary tree in which elements are added only in one direction

2.5.2 AVL Tree

- An AVL tree is a self-balancing binary search tree, in which the heights of the two child subtrees of any node differ by atmost one.
- Insertions and deletions may require the tree to be rebalanced by one or more tree rotations.

For GATE DA Crash Course, visit: www.piyushwairale.com

- The balance factor of a node is the height of its left subtree minus the height of its right subtree (sometimes opposite) and a node with balance factor—1, 0 or -1 is considered balanced. A node with any other balance factor is considered unbalanced and requires rebalancing the tree.
- The balance factor is either stored directly at each node or computed from the heights of the subtrees.

2.5.3 Binary Heap

- A binary heap is a heap data structure created using a binary tree. It can be seen as a binary tree with two additional constraints.
- The shape property: The tree is a complete binary tree; that is, all levels of the tree, except possibly the last one (deepest) level of the tree is not complete, the nodes of that level are filled, from left to right.
- **Max-Heap**

  A heap in which each node is greater than or equal to its children is called max-heap. Max-Heap generally used for heap sort.
- **Min-Heap**

  A heap in which, each node is smaller than or equal to its children is called Min-Heap. Min-heap generally used to implement priority queue.

  Note: By default heap represent Max-Heap

2.6 Hash Table

- A Hash table is defined as a data structure used to insert, look up, and remove key-value pairs quickly.
- It operates on the hashing concept, where each key is translated by a hash function into a distinct index in an array. The index functions as a storage location for the matching value. In simple words, it maps the keys with the value.
- Hash tables are a type of data structure in which the address or the index value of the data element is generated from a hash function. That makes accessing the data faster as the index value behaves as a key for the data value. In other words Hash table stores key-value pairs but the key is generated through a hashing function.
- So the search and insertion function of a data element becomes much faster as the key values themselves become the index of the array which stores the data.
- In Python, the Dictionary data types represent the implementation of hash tables. The Keys in the dictionary satisfy the following requirements.

For GATE DA Crash Course, visit: www.piyushwairale.com

- The keys of the dictionary are hashable i.e. the are generated by hashing function which generates unique result for each unique value supplied to the hash function.
- The order of data elements in a dictionary is not fixed.

For GATE DA Crash Course, visit: www.piyushwairale.com

References

- Pearson's GATE CS Notes
- geeksforgeeks.org/
- javatpoint.com/
- w3schools.com/
- tutorialspoint.com/
- byjus.com

For GATE DA Crash Course, visit: www.piyushwairale.com